AIAC-ASSIGNMENT-19.3

NAME:ESHWAR NOMULA BATCH : 32 HNO:2303A52072

Task 1 – Python to C++ Conversion

Task:

Translate a simple Python class into C++ using AI assistance.

Instructions:

• Prompt AI to convert a Python class representing a Student with

attributes and methods into equivalent C++ code

• Ensure proper handling of:

o Constructors

o Data types

o Access specifiers

• Compile and run both versions to verify output consistency

Expected Output:

An equivalent working C++ class translated from Python.

```python
# generate a Python code with a class representing a Student with attributes
and methods
from numpy import std


class Student:
    def __init__(self, name, age, grade):
        self.name = name
        self.age = age
        self.grade = grade

    def get_info(self):
        return f"Name: {self.name}, Age: {self.age}, Grade: {self.grade}"

    def is_passing(self):
        return self.grade >= 60

# Example usage
student1 = Student("Alice", 20, 85)
print(student1.get_info())  # Output: Name: Alice, Age: 20, Grade: 85
print(student1.is_passing())  # Output: True
```

```cpp
# convert the above python code into C++
#include <iostream>
#include <string>
using namespace std;
class Student {
private:
    string name;
    int age;
    int grade;
public:
    Student(string n, int a, int g) {
        name = n;
        age = a;
        grade = g;
    }
    string getInfo() {
        return "Name: " + name + ", Age: " + to_string(age) + ", Grade: " +
to_string(grade);
    }
    bool isPassing() {
        return grade >= 60;
    }
};
int main() {
    Student student1("Alice", 20, 85);
    cout << student1.getInfo() << endl; // Output: Name: Alice, Age: 20,
Grade: 85
    cout << (student1.isPassing() ? "True" : "False") << endl; // Output: True
    return 0;
}
```

Task 2 – Java to Python Function Conversion

Task:

Convert a Java method that checks whether a number is prime into

Python.

Instructions:

- **Ask AI to translate a Java isPrime() method into Python**
- **Test the function with multiple inputs**
- **Ensure the Python version follows Pythonic syntax and logic**

Expected Output:

A correct Python function equivalent to the original Java prime-

checking method.

```
# generate a java code with a isprime(int n) methodpublic class PrimeChecker {
    from xmlrpc.client import boolean


public static boolean isPrime(int n) {
        if (n <= 1) {
            return false;
        }
        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0) {
                return false;
            }
        }
        return true;
    }

# convert the above java code into python
def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
```

Task 3 – Pseudocode to Python Implementation

Task:

Translate a given pseudocode algorithm into Python using AI.

Instructions:

- **Provide AI with pseudocode for sorting numbers (e.g., bubble**

sort or selection sort)

- **Ask AI to convert it into executable Python code**
- **Validate correctness using sample input lists**

Expected Output:

A working Python program that correctly implements the pseudocode

logic.

```
# convert the given bubblesort pseudocode into python code
# Pseudocode:
# procedure bubbleSort(A : list of sortable items)
#     n = length(A)
#     repeat
#         swapped = false
#         for i = 1 to n-1 inclusive do
#             if A[i-1] > A[i] then
#                 swap(A[i-1], A[i])
#                 swapped = true
#     until not swapped
def bubble_sort(arr):
    n = len(arr)
    while True:
        swapped = False
        for i in range(1, n):
            if arr[i - 1] > arr[i]:
                arr[i - 1], arr[i] = arr[i], arr[i - 1]
                swapped = True
        if not swapped:
            break
    return arr
```

Task 4 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in

Python.

Instructions:

- **Provide AI with a SQL query (e.g., SELECT name, salary FROM**

employees WHERE salary > 50000).

- **Ask AI to generate equivalent Pandas code.**

- **Test the generated code on a sample DataFrame.**

Expected Output:

- **Equivalent Pandas query that retrieves the correct subset of data.**

```
# convert given sql query into pandas code
#SELECT name, salary
#FROM employees
#WHERE salary > 50000;
import pandas as pd
```

```
# Assuming we have a DataFrame 'employees' with columns 'name' and 'salary'
employees = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie', 'David'],
    'salary': [60000, 45000, 70000, 30000]
})
result = employees[employees['salary'] > 50000][['name', 'salary']]
print(result)
```

Task 5 – Real-Time Application: Algorithm Translation Across

Languages

Scenario:

A company maintains algorithms written in different programming

languages.

Examples:

• Converting a Python searching algorithm into C

• Translating a Java sorting program into JavaScript

Instructions:

• Use AI to translate a selected algorithm between two

programming languages

• Execute and test both versions

• Document translation challenges such as:

o Syntax differences

o Library support

o Memory management

Expected Output:

Equivalent, tested code implementations in two different programming

languages.

```
# generate linear search algorithm in python
from py_compile import main


def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
```

```
            return i
    return -1
# Example usage
arr = [1, 2, 3, 4, 5]
target = 3
result = linear_search(arr, target)
if result != -1:
    print(f"Element found at index: {result}")
else:
    print("Element not found in the array")

# convert the above linear search algorithm into C
#include <iostream>
using namespace std;
int linearSearch(int arr[], int size, int target) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == target) {
            return i;
        }
    }
    return -1;
}
int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int size = sizeof(arr) / sizeof(arr[0]);
    int target = 3;
    int result = linearSearch(arr, size, target);
    if (result != -1) {
        cout << "Element found at index: " << result << endl;
    } else {
        cout << "Element not found in the array" << endl;
    }
    return 0;
}
```